

xHumanPro

Guida operativa - Piattaforma di ottimizzazione campagne per farmacie e-commerce

Versione: 1.0 (FASE 1 cross-channel + FASE 3 in build)

Data: 23 maggio 2026

Owner: Stefano Quitadamo (xHumanPro / Armah srl)

Stack: Node.js 20 + PostgreSQL 16 + React 18 + Docker

Tenant live: 8 farmacie (Farmacri, Mandanici, Papa, SubitoFarma, Procaccini, Ospedale, MPF, San Vito)

Indice

1. Mission e contesto di business
2. Architettura tecnica
3. Modello multi-tenant e onboarding
4. Pipeline dati e cron
5. Health Engine
6. Feed Engine e quarantena
7. Regole prezzo Farmabooster
8. SKU Margins e moduli di ottimizzazione
9. Supervisor Agent
10. Cross-tenant intelligence
11. Merchant Center
12. Cross-Channel Analysis (FASE 1)
13. Google Ads integration (FASE 3)
14. Deploy, observability, sicurezza

1. Mission e contesto di business

Cos'è xHumanPro

xHumanPro è una piattaforma SaaS multi-tenant che orchestra l'ottimizzazione delle campagne **Trovaprezzi.it** e **Google Shopping** per una rete di farmacie e-commerce italiane gestite con Magento + Farmabooster.

Il sistema non sostituisce Farmabooster: lo affianca, leggendo il pricing applicato (regole Ricarico / Sconto / Salva Bilancio / Muro) e aggiungendo un layer di ottimizzazione basato su:

- **Margine reale per SKU** al netto di COGS, spedizione, CPC Trovaprezzi, IVA
- **Attribuzione GA4** dei click TP all'effettivo fatturato store
- **Posizionamento competitivo** via scraping comparatori e Merchant Center
- **Cross-tenant intelligence**: ciò che funziona su un seller informa gli altri

Il problema risolto

Farmabooster gestisce 100k+ SKU ma tratta il feed Trovaprezzi come on/off (civetta = 1 o 0). Senza un layer sopra accade che:

- Prodotti civetta che generano click ma **non convertono** bruciano budget (incidenza TP del 8-12% sul fatturato vs target 4-5%)
- Prodotti civetta che convertono **sotto floor MOL 15%** producono ordini ma erodono marginalità
- Prodotti potenzialmente "pepite" (non civetta ma con domanda + margine alto) restano invisibili
- Regole di Farmabooster (Muro, Salva Bilancio) applicate in modo statico mancano sweet spot stagionali e cross-seller

North Star e guard rail

North Star: fatturato store mensile per tenant.

Guard rail: incidenza costo TP / fatturato store $\leq 5\%$, MOL store $\geq 20\%$ (floor minimo 15%).

Mai tagliare feed o spesa se questo riduce fatturato netto: il fatturato comanda, l'incidenza è il vincolo.

Obiettivi strategici 2026

1. **Revenue up**: tornare ai picchi 2024-2025 sui tenant flat
2. **Cost down**: ridurre la spesa TP del 25-30% senza intaccare fatturato
3. **MOL $\geq 20\%$** : spingere tutti i tenant sopra il floor (oggi MPF, Mandanici sotto)

4. **Diretto over Grossista:** incentivare sourcing diretto/farmacia vs grossista (margine +3-7%)

2. Architettura tecnica

Stack

Layer	Tecnologia	Note
Backend	Node.js 20 LTS + Express 4	Singolo monolite, no microservizi
Database	PostgreSQL 16	Multi-tenant logico (tenant_id colonna)
Frontend	React 18 + Vite + TailwindCSS	SPA, autenticazione JWT + TOTP
Container	Docker Compose / Warden	backend, db, frontend, traefik
Scheduling	setInterval interni	No cron esterno, tutto in-process
Scraping	puppeteer-core + Chromium	Trovaprezzi analytics CSV, comparatori
Storage allegati	Google Drive (Service Account)	CSV scraping, report PDF
Alerting	Telegram Bot + SMTP	KPI alert, daily summary, cron failure
AI	Anthropic Claude (anthropic-ai/sdk)	Agent SDK per supervisor decisions

Componenti principali

```
backend/  
  routes/    endpoint REST (orders, products, trovaprezzi, optimization, ab-tests,  
              cross-channel, pareto, rule-optimizer, agent, supervisor, onboarding)  
  services/  business logic: health, feed, pricing, quarantine, GA4, MC, sku margins,  
              cross-tenant, ab tests, supervisor agent, optimization log  
  db/migrations/ 001..027 SQL migrations, idempotenti, applicate al boot  
  middleware/  auth (JWT), acl (role-based)  
  
frontend/src/  
  pages/      Dashboard, Products, Orders, FeedOptimizer, Trovaprezzi, Statistiche,  
              ParetoRule, RuleOptimizer, CrossChannel, Agent, Onboarding,  
              TenantConfig, AdminUsers, AdminTenants  
  components/  Layout, ProtectedRoute, RoleGuard  
  contexts/    AuthContext (JWT + tenant selection)
```

Flusso dati ad alto livello

Magento (orders, products)
Farmabooster (price rules)
Trovaprezzi (clicks CSV) > [Ingest cron] > PostgreSQL
Drive (scraper competitor) > [Health Engine]
GA4 (attribution) > [Feed Engine]
Merchant Center > [Optimization modules]
Google Ads (FASE 3) > [Supervisor agent]
> [Cross-tenant]

Push back actions:

- Magento (price, civetta)
- Farmabooster (markup, tolerance)
- Google Ads bid (FASE 3)

3. Modello multi-tenant e onboarding

Schema dati

Tutte le tabelle dati hanno `tenant_id UUID` con foreign key su `tenants(id)` e cascade delete. Nessun database fisicamente separato: isolamento logico via filtro `WHERE tenant_id = $1` applicato in ogni query.

Tabella tenants	Campo	Uso
	id (UUID)	chiave primaria
	name	label leggibile (Papa, Procaccini, ...)
	status	active / paused / archived
	magento_base_url	endpoint REST Magento
	farmabooster_api_url	endpoint FB
	xhumanpro_start_date	data prima azione, baseline pre-xHuman

Tenant configs (key-value cifrato)

Credenziali per-tenant in `tenant_configs(tenant_id, config_key, config_value)`. Valori sensibili cifrati AES-256-GCM via `services/crypto.js`.

config_key	Tipo	Cosa contiene
magento_token	cifrato	access token API Magento
farmabooster_username / password	cifrato	credenziali Farmabooster
trovaprezzi_username / password	cifrato	login TP analytics
trovaprezzi_shipping_cost	plain	spese spedizione effettive (€)
ga4_property_id	cifrato	GA4 property numerico
google_merchant_id	cifrato	Merchant Center account ID
google_ads_customer_id	cifrato	Customer ID Google Ads (FASE 3)
scraper_drive_folder_id	cifrato	folder Drive con CSV scraping
skumargins_overlay_disabled	plain	kill-switch overlay margine (es. Mandanici)

Global config

Credenziali e risorse condivise vivono in `global_config(config_key, config_value)`: service account Google Drive/MC/Ads, Telegram bot token, Anthropic API key, developer token Google Ads.

Onboarding di un nuovo tenant

1. **POST /api/admin/tenants** crea record tenant

2. **Configurazione** via UI: Magento URL+token, Farmabooster URL+credenziali, TP login, GA4 property, MC account, Drive folder
3. **SSE-driven import iniziale** via `POST /api/onboarding/run` :
 - Test connettività Magento / Farmabooster / Drive / Telegram
 - Import storico ordini (180 giorni)
 - Import catalogo Farmabooster (prodotti + price rules)
 - Import storico TP (30 giorni)
 - Import Merchant Center (catalog + statuses)
 - Computazione baseline pre-xHuman (30 giorni prima)
4. **xhumanpro_start_date** impostata al completamento; tutto ciò che viene dopo è "post-xHuman"

SSE heartbeat: l'onboarding SSE invia ping ogni 20s perché nginx chiude le connessioni idle a 60s. Senza heartbeat l'import lungo (oltre i 60s) si interrompe lato client.

4. Pipeline dati e cron

Tutto orchestrato in-process da `backend/services/healthCron.js`. Nessun cron di sistema, nessun job queue esterno: setInterval con flag `_cronRunning` per evitare overlap.

Cron attivi

Cron	Frequenza	File	Cosa fa
OrderSync	ogni 60min	orderSync.js	Import ordini Magento ultimi 15gg, stagger 60s tra tenant
ProductSync	ogni 360min	productSync.js	Import catalogo Farmabooster, price rules
ZombieCron TP	ogni giorno 06:00	zombieCron.js	Download CSV TP analytics (puppeteer + login), persist click giornalieri
HealthCron	ogni 60min	healthCron.js	Full pipeline: scraper import, health scores, GA4, MC (notte), feed engine, predictions, reactivations
AlertMonitor	ogni 2h	alertMonitor.js	Verifica freshness di ogni loop, Telegram alert se fermo
DailySummary	ogni giorno 09:00	kpiAlerts.js	Telegram summary del giorno prima per ogni tenant

HealthCron - dettaglio pipeline oraria

1. **Step 0a** - Zombie clicks today (puppeteer TP analytics login + CSV download)
2. **Step 0b** - Scraper import (global, una sola volta per ciclo, ~100k+ entries)
3. **Step 1** - Per ogni tenant: sync civetta Magento DB
4. **Step 2** - Health scores (productHealth: scoring + classificazione)
5. **Step 2.1** - GA4 attribution (se credenziali presenti)
6. **Step 2b** - **NUOVO** Merchant Center sync (finestra 04-06, 1x/giorno, gate 22h)
7. **Step 3** - Daily data collection (tracking, killers identification, snapshot incidenza)
8. **Step 4** - Feed engine (decisioni REMOVE/ADD/PRICE_CUT/KEEP/MONITOR)
9. **Step 5** - Predictions + monitoring + calibration
10. **Step 6** - Reactivations check (quarantena scaduta?)
11. **Step 7** - Observation windows (recidivism detection)
12. **Step 8** - Competitor snapshot
13. **Step 9** - Stable cache rebuild (per Farmabooster External API)
14. **Step 10** - Supervisor fast-path checks slow-path se RED/YELLOW

Job giornalieri / settimanali (in-line in HealthCron)

- **Basket Analysis** notte 03-05 (computeBasketForAll, per cross-sell traini)
- **Rule Optimizer** lunedì 07-09 (settimanale, propone modifiche regole FB)

- **Quarantine Recovery** daily (riapre SKU che si comportano meglio post-pausa)
- **Cross-tenant intelligence** settimanale
- **Recommendation Verifier** orario (riconcilia raccomandazioni applied)
- **A/B Tests Evaluator** orario (chiude test in scadenza, applica rollback)

5. Health Engine

Il cuore analitico. Per ogni SKU civetta calcola un set di score normalizzati, una classificazione e una raccomandazione.

Score (0-100)

- `revenue_score` - fatturato store ultimi 30gg vs media tenant
- `growth_score` - trend ultimi 7gg vs 30gg
- `efficiency_score` - vendite / click TP (conversion ratio penalizzato da costo)
- `competitive_score` - posizione scraping (prima posizione = 100, fuori top 10 = 0)
- `indirect_score` - GA4 assisted revenue (acquisti da altri canali post-vista TP)
- `seasonal_score` - allineamento al pattern stagionale del season_id

Composito: `health_score` media pesata dei sopra.

Classificazione SKU

Classe	Definizione	Decisione tipica
STAR	MOL stabile sopra baseline + revenue solido	KEEP, protetto da quarantena
WIN	MOL sopra baseline, revenue ok	KEEP
OK	Trend stabile	MONITOR
WATCH	MOL o revenue in calo (uno solo)	MONITOR + price adj
LOSER	Entrambi sotto baseline	PRICE_CUT o REMOVE
BURNER	Click 30gg ma 0 ordini 60gg	QUARANTINE
COLD	0 click 0 ordini	nessuna decisione

Inputs critici

- `products.erp_cost` - costo grossista da Farmabooster (~71% catalogo)
- `products.erp_cost_imputed` - imputazione cost via avg brand (29% senza erp_cost)
- `order_items.row_total_incl_tax` - revenue IVA-inclusa allineato a erp_cost
- `health_config.avg_tp_cpc` - CPC tenant-specific (€0.2773 media reale, vs €0.27 default)
- `tenant_configs.trovaprezzi_shipping_cost` - spedizione effettiva (esclusa per ordini Ritirato/ritiro_farmacia)

6. Feed Engine e quarantena

Trasforma gli output Health Engine in azioni concrete sul feed Trovaprezzi. Una decisione per SKU per ciclo, scritta in `feed_actions`.

Azioni

Action	Quando	Effetto
KEEP	SKU star/win, conferma posizione	nessun cambio
ADD	SKU civetta=0 ma con domanda e margine	marca civetta=1 in Magento
REMOVE	burner / loser senza recovery	marca civetta=0
PRICE_CUT	perso posizione con margine ancora positivo	set price ribassato (rispetta floor MOL)
MONITOR	watch, troppo presto per decidere	nessun cambio, ri-eval prossimo ciclo
RATE_LIMITED	già modificato di recente	skip
STAR_PROTECTED	SKU star, immune da quarantena	force KEEP
MARGIN_QUARANTINED	MOL sotto floor su revenue alto	quarantena per N giorni
CROSS_TENANT_REFINED	peer tenant ha trovato config migliore	applica peer config

Quarantena progressiva

Migration 010. Un SKU che entra in quarantena multiple volte non torna mai attivo permanentemente: la finestra di osservazione aumenta esponenzialmente (3gg 7gg 14gg 30gg permanent).

La quarantena ha un budget per-SKU dinamico calcolato come $\text{margine_eur} / \text{CPC_tenant}$: oltre quel numero di click senza ordine, lo SKU va in pausa. Niente soglia fissa "20 click = pausa".

Cross-tenant guard

PRICE_CUT cross-tenant safety: quando un seller della rete xHumanPro abbassa il prezzo, il floor è il *minimo competitor esterno*, escludendo gli altri seller xHumanPro. Evita guerre di prezzo interne che cannibalizzano la rete.

7. Regole prezzo Farmabooster

Farmabooster applica 4 macro-tipi di regola, gestite via tabella `price_rules` con campo `rule_data` JSONB. xHumanPro non sostituisce le regole: ne propone modifiche tramite Rule Optimizer.

Macro-regole

Tipo	Logica	Quando si usa
Ricarico	$\text{price} = \text{erp_cost} \times (1 + \text{markup_pct})$	SKU diretti / farmacia
Sconto	$\text{price} = \text{reference} \times (1 - \text{sconto_pct})$	promozioni temporanee
Salva Bilancio	$\text{price} = \text{MAX}(\text{ricarico}, \text{prezzo competitor} + \text{delta})$	SKU dove la marginalità è critica
Muro	$\text{price} = \text{competitor_best_price} + \text{tolerance}$	SKU dove la competitività comanda

Floor MOL applicato sopra tutte le regole

Fascia prezzo	Markup minimo
fino a €10	18%
€10 - €30	14%
oltre €30	12%

Rule Optimizer (settimanale)

Lunedì 07-09 analizza per ogni tenant:

- Regole con incidenza > 15% in fascia critica propone `markup_up` o `quarantine_burner`
- Regole "Muro" troppo aggressive su SKU bassa-conversione propone `shipping_exclude`
- Pattern weekend (incidenza sabato +50% vs giovedì) propone `sub_segment_deep`
- SKU con peer-tenant performance migliore propone `cross_tenant_leverage`

Le raccomandazioni sono scritte in `rule_optimizer_recommendations` e validate dal Recommendation Verifier orario.

Regola d'oro: prima di proporre modifiche, leggere sempre `price_rules.rule_data` (mai dedurre dai nomi o hardcoded). I default cambiano per tenant.

8. SKU Margins e moduli di ottimizzazione

skuMargins.js - la fondazione

Per ogni SKU civetta calcola il margine netto reale su 4 finestre: *pre-xHumanPro* (30gg prima del go-live), *ultimi 30gg*, *15gg*, *7gg*.

```
gross_margin = revenue_incl_tax - cogs (qty × erp_cost_imputed)
shipping_alloc = ordini × shipping_cost (escluso per Ritirato/ritiro_farmacia)
tp_cost = click_TP × CPC_tenant
net_margin = gross_margin - shipping_alloc - tp_cost
mol_pct = net_margin / revenue × 100
```

Verdetto comparando le 3 finestre recenti vs baseline pre-xHuman: star / win / ok / watch / loser / burner / cold.

Optimization Modules (migration 014)

Modulo	Obiettivo
obj_revenue_up	Aumenta fatturato TP
obj_cost_down	Riduce costo TP
obj_mol_protect	Protegge MOL ≥ 20%
obj_diretto_priority	Prioritizza sourcing diretto/farmacia

Basket Analysis

Notte 03-05. Calcola per ogni SKU il valore "traino": quante volte appare in carrelli con altri SKU venduti a margine pieno. Output usato dal Rule Optimizer per identificare loss leader utili (perdita su SKU X compensata da cross-sell SKU Y).

A/B Test framework (migration 025)

Sistema per testare modifiche di posizionamento controllate.

- **Candidati:** SKU con margine alto + posizione 1 TP + conversion rate basso (es. ORALB)
- **Test:** alza prezzo a +€0.01 sopra competitor #2, lasciando posizione 2-3 invece di 1
- **Safety:** max +5% rispetto a prezzo iniziale, target solo competitor sopra il nostro
- **Decision thresholds:** success_ratio 0.7 (mantiene 70% ordini) conferma, rollback_ratio 0.5 (perde 50%) rollback automatico
- **Evaluator orario:** decide a fine finestra test

9. Supervisor Agent

Layer di sicurezza che osserva l'output di Health/Feed Engine e blocca decisioni rischiose o segnala anomalie.

Fast path (orario)

Check leggeri:

- Pipeline freshness (orderSync < 24h? zombie < 48h?)
- Incidenza tenant < soglia critica?
- MOL store < floor 15%?
- Numero quarantene oggi vs media (spike anomalo?)
- Drift CPC vs storico

Verdict: GREEN / YELLOW / RED.

Slow path (Claude agent)

Triggerato solo se fast-path emette RED o YELLOW persistente. Invoca Claude Sonnet 4.6 via Anthropic SDK con:

- Stato corrente tenant (KPI + ultime decisioni)
- Trend ultimi 30gg
- Context business (regole FB, baseline pre-xHuman)

L'agent può: aprire ticket, marcare decisioni pending per revisione manuale, raccomandare rollback specifici. Mai esegue automaticamente azioni distruttive.

Anti-false-alarm: per emettere RED servono multi-evidence (incidenza + MOL + freshness coerenti). Confronti su partial-day soppressi. False negativo > false positivo: meglio non allertare che spammare.

Weekly review

Domenica 18:00. Riassunto settimanale per-tenant via Telegram: fatturato vs target, incidenza, top 5 burner attivati, top 5 reactivation success, decisioni Rule Optimizer applicate.

10. Cross-tenant intelligence

Migration 013. La rete xHumanPro condivide pattern senza condividere dati identificabili.

Logica

1. Per ogni SKU comune a più tenant (matching su EAN o sku normalizzato), confronta performance: conversion rate, MOL, posizione, click-to-order
2. Identifica i tenant "peer leader" per quello SKU
3. Propone agli altri tenant la configurazione del leader (markup, position target, civetta on/off)

Esempio reale

Papa e San Vito hanno 6.803 SKU in overlap con prezzi *identici*, ma San Vito converte 2x meglio. Diagnosi: la differenza non è il pricing ma la *reputation seller* (San Vito ha più recensioni TP). Cross-tenant non clona il prezzo (già uguale) ma segnala a Papa di lavorare sulla reputation, non sul margine.

Safety rules

- **No internal pricing war:** floor PRICE_CUT è il min competitor *esterno*
- **Confidence weighted:** raccomandazioni cross-tenant attivate solo con $n \geq 30$ ordini peer su SKU
- **Tenant opt-out:** tenant_configs.cross_tenant_disabled=true salta il modulo

11. Merchant Center

Migration 006. Ingest dati Google Shopping organico via Content API v2.1.

Cosa importiamo

Endpoint API	Cosa	Frequenza
products.list	Catalogo offerte: title, brand, price, availability, customLabels	daily
productstatuses.list	approval_status (approved/disapproved/pending) per offer_id	daily
productPricingReports	Benchmark prezzo vs competitor	daily
productPriceCompetitiveness	Suggerimenti prezzo per ranking	daily
productView	Click potential rank	daily
productPerformanceView	Daily impressions, clicks, CTR, conversions	daily

Loop automatico

Hook in HealthCron step 2b. Finestra **04:00-06:00**, gate 22h per-tenant. Pesante (~60k SKU per Farmacri = 12 min). Errori isolati: se MC fallisce non blocca il resto della pipeline.

Bug fix recenti

- **Dedup bug:** ON CONFLICT su `(date, offer_id)` falliva quando lo stesso offer_id appariva due volte nel payload (caso multi-channel). Fix: dedupMap pre-insert.
- **Approval status NULL:** parsing di `productId` assumeva prefisso `online:it:IT:` ma alcuni tenant hanno `online:it:-:`. Fix: `productId.split(':')[0].pop()`.

12. Cross-Channel Analysis (FASE 1)

NUOVO - rilasciato 23/05/2026: tab Cross-Channel in dashboard che incrocia Merchant Center, GA4, vendite Magento e click Trovaprezzi per identificare anomalie sfruttabili a livello SKU.

Service crossChannelAnalysis.js

Per ogni SKU del tenant, finestra 7/14/30/60/90gg, calcola:

Campo	Fonte
approval_status	merchant_center_products
mc_impressions / clicks / ctr	merchant_center_daily aggregato
mc_conversions / conv_value	merchant_center_daily
tp_clicks	zombie_clicks aggregato
orders_count / qty_sold / revenue	order_items + orders
cogs / mol_pct	erp_cost COALESCE imputed
ga4_tp_purchases / revenue	product_health_scores

Anomalie rilevate

Anomaly code	Definizione	Azione suggerita
wasted_serving	MC approved + impressioni > 500 + 0 click	Bid troppo basso o titolo/immagine debole. Verifica feed.
feed_blocked	disapproved/pending ma ordini reali presenti	Fix policy violation Merchant Center: si stanno perdendo ordini Shopping.
loss_leader	MC clicks > 20 + revenue > 100 + MOL < 15%	Alza markup oppure quarantena se non c'è cross-sell.
dead_stock_mc	30gg approved + 0 vendite + impressioni > 100	Pause MC, prodotto inerte.
high_ctr_low_cvr	CTR > 5% + 0 conversioni MC + click > 30	Problema landing/prezzo: gente clicca ma esce.

Endpoint

```
GET /api/cross-channel/summary?days=30
-> KPI aggregati: sku_total, mc_approved/disapproved/pending,
    revenue_total, revenue_with_anomaly, anomalies_by_code, top_anomalies

GET /api/cross-channel/products?days=30&anomaly=loss_leader
-> tutte le righe per-SKU con flag anomaly
```

UI

Pagina `/cross-channel` in sidebar: 5 KPI card in alto, chip filtri per anomaly code, tabella SKU completa con badge severity. Filtro temporale 7/14/30/60/90gg. Auto-refresh al cambio tenant.

13. Google Ads integration (FASE 3)

Status: in build. Developer token Test Access ottenuto (23/05/2026). Basic Access richiesto, ETA 1-2 settimane. Codice deployabile come Test Access su account di prova, switch a produzione senza modifiche dopo approvazione.

Perché Google Ads

Trovaprezzi è ottimizzato ma Google Shopping (paid via Ads) genera 30-50% del traffico paid sui nostri tenant senza visibilità nostra: vediamo solo l'aggregato GA4. Manca:

- Costo per SKU su Ads
- Impressioni / click per product group
- Search terms che convertono / che bruciano budget
- Possibilità di bid adjustment automatico basato su margine reale

Architettura prevista

Componente	Cosa fa
Migration 027_google_ads.sql	Tabelle: google_ads_campaigns_daily, google_ads_products_daily, google_ads_search_terms, google_ads_runs
services/googleAdsService.js	Client autenticato con dev_token + service account, import giornaliero metrics
HealthCron step 2c	Sync Ads in finestra 05:00-07:00 (dopo MC)
Routes /api/google-ads	summary, products, campaigns, search-terms, bid-suggestions
UI tab "Google Ads"	Estensione del Cross-Channel con colonne Ads
Bid optimizer	Computa CPC ottimale per product group = $MOL_{eur} \times target_{roas} / conversion_{rate}$, scrive back via API

Setup operativo richiesto

1. Developer token Google Ads (**ottenuto come Test**, richiesto Basic Access)
2. Service account autorizzato per Ads API (riusiamo quello di Drive/MC con scope esteso)
3. Customer ID Google Ads per tenant (`tenant_configs.google_ads_customer_id`) - da raccogliere dal MCC
4. Login customer ID = MCC ID nel `login_customer_id` header delle richieste API
5. Privacy policy pubblica su armah.it (richiesta da Google per Basic Access review)

Metriche target da importare

```
SELECT
  campaign.id, campaign.name, campaign.status,
  segments.date,
  metrics.impressions, metrics.clicks, metrics.cost_micros,
  metrics.conversions, metrics.conversions_value,
  metrics.ctr, metrics.average_cpc
FROM campaign
WHERE segments.date DURING LAST_30_DAYS
```

```
SELECT
  segments.product_item_id,
  segments.date,
  metrics.impressions, metrics.clicks, metrics.cost_micros,
  metrics.conversions, metrics.conversions_value
FROM shopping_performance_view
WHERE segments.date DURING LAST_30_DAYS
```

Roadmap di rilascio

1. **Settimana 1** (dopo Basic Access): migration + service + ingest read-only su 1 tenant pilota (Farmacri)
2. **Settimana 2**: estensione a tutti i tenant + colonne Ads in Cross-Channel UI
3. **Settimana 3**: bid suggestions calculate (read-only), confronto manuale
4. **Settimana 4**: auto-apply bid changes (con safety cap +/-15% per ciclo, rollback su drop conversion > 30%)

14. Deploy, observability, sicurezza

Deploy

Step	Comando
Sync code	git push (deploy attivato manualmente)
Rebuild backend	<code>docker compose up -d --build backend</code>
Migrations	auto al boot, idempotenti (IF NOT EXISTS)
Frontend	build statico <code>npm run build</code> servito da nginx
Rollback	checkout commit precedente + rebuild

Observability

- **Logs:** stdout container, `docker compose logs backend`
- **Alert SMTP:** alertMonitor stefano@fix.one via smtp2go
- **Alert Telegram:** KPI critici, daily summary 09:00, weekly review domenica 18:00
- **Health endpoint:** `GET /api/health` ritorna 200 se backend up
- **Cron freshness:** AlertMonitor controlla ultimi ordini < 24h, click TP < 48h, health < 4h, feed < 4h

Sicurezza

- **Auth:** JWT con expiry 8h, TOTP obbligatorio per superadmin/admin
- **ACL:** middleware requireRole(superadmin | admin | viewer)
- **Crittografia at-rest:** tutti i secrets in tenant_configs e global_config cifrati AES-256-GCM
- **Rate limiting:** apiQueue interna per chiamate Farmaboster / Magento (timeout 180s)
- **Process safety:** process.on('unhandledRejection') + ('uncaughtException') con notify Telegram, no crash silente

Backup

- Postgres dump giornaliero su Drive (script esterno)
- Migration SQL versionate in git
- Config tenant in tabella, esportabile

Contatti

Stefano Quitadamo - me@stefanoquitadamo.com / stefano@fix.one
Repository: [git@github.com:helgvar/xhuman-pro.git](https://github.com/helgvar/xhuman-pro.git)

Guida generata automaticamente il 23/05/2026 - xHumanPro v1.0